

Berkane Lab — Python Intensive Manual

M. Haithem BERKANE

Bilingual course companion · English edition

This manual contains ten chapters, one hundred worked examples, and one hundred and fifty solved exercises. The situations are inspired by the supplied science, conditions, and loops exercise collections: physics, mathematics, biology, sport, chemistry, and data reasoning.

Python foundations

Worked examples

Example 1 — 01 · Display

Situation. An association displays a welcome message.

```
prenom = "Lina"  
print("Bienvenue", prenom)
```

Explanation. print assembles the message and variable.

Example 2 — 02 · Conversion

Situation. A typed age must enter a calculation.

```
age = int("16")  
print(age + 2)
```

Explanation. int turns text into an integer.

Example 3 — 03 · Price

Situation. Two prices are added for an order.

```
a = float("12.5")
b = float("7.5")
print(a + b)
```

Explanation. float enables decimal calculation.

Example 4 — 04 · Type

Situation. We check a value type.

```
valeur = 12
print(type(valeur).__name__)
```

Explanation. type describes the value, not its name.

Example 5 — 05 · Safe input

Situation. A conversion is explained before display.

```
texte = "42"
nombre = int(texte)
print(nombre, nombre * 2)
```

Explanation. The same data can be text then number after conversion.

Example 6 — 01 · Average speed

Situation. A cyclist covers 42 km in 2 h.

```
distance = 42
duree = 2
print(distance / duree)
```

Explanation. Average speed is distance divided by time.

Example 7 — 02 · Kelvin

Situation. A lab converts 25°C to kelvins.

```
celsius = 25
kelvin = celsius + 273.15
print(kelvin)
```

Explanation. The conversion adds 273.15 without modifying the initial value.

Example 8 — 03 · Energy

Situation. Potential energy is estimated from mass, g, and height.

```
masse = 2
g = 9.81
hauteur = 3
print(masse * g * hauteur)
```

Explanation. Each variable carries a unit and the formula is translated directly.

Example 9 — 04 · Heart rate

Situation. A session calculates a theoretical maximum heart rate.

```
age = 16
fcm = 220 - age
print(fcm)
```

Explanation. The formula links an input value to a constant.

Example 10 — 05 · Ohm law

Situation. A circuit knows voltage and resistance.

```
tension = 12
resistance = 6
intensity = tension / resistance
print(intensity)
```

Explanation. intensity receives the quotient U/R.

Solved exercises

Exercise 1 — 01 · City

Task. Ask for a city and display a welcome.

```
ville = input("Ville : ")
print("Bienvenue à", ville)
```

Correction. input provides text; print displays.

Exercise 2 — 02 · Age

Task. Ask for age and add 1.

```
age = int(input("Age : "))
print(age + 1)
```

Correction. Convert before calculating.

Exercise 3 — 03 · Celsius

Task. Convert 20°C to Fahrenheit.

```
c = 20
print(c * 9 / 5 + 32)
```

Correction. Apply the formula after assignment.

Exercise 4 — 04 · Rectangle

Task. Calculate an area.

```
longueur, largeur = 5, 3  
print(longueur * largeur)
```

Correction. Area is a product.

Exercise 5 — 05 · Remainder

Task. Find remainder of 17 by 5.

```
print(17 % 5)
```

Correction. % gives the remainder.

Exercise 6 — 06 · Text

Task. Combine first name and city.

```
print("Lina" + " · Oran")
```

Correction. + concatenates text.

Exercise 7 — 07 · Rounding

Task. Round a price.

```
print(round(12.567, 2))
```

Correction. round limits decimal places.

Exercise 8 — 08 · Boolean

Task. Check if 12 is positive.

```
print(12 > 0)
```

Correction. The comparison returns True.

Exercise 9 — 09 · Swap

Task. Swap two variables.

```
a, b = 2, 5  
a, b = b, a
```

Correction. Python supports multiple assignment.

Exercise 10 — 10 · Output

Task. Display a readable result.

```
score = 15  
print("Score :", score, "/ 20")
```

Correction. Commas separate displayed items.

Exercise 11 — 11 · Flow rate

Task. Calculate the flow rate of 10 L in 5 min.

```
volume = 10  
duree = 5  
print(volume / duree)
```

Correction. Flow rate is volume divided by time.

Exercise 12 — 12 · Density

Task. Calculate density for 250 g in 100 cm³.

```
masse = 250
volume = 100
print(masse / volume)
```

Correction. The formula $\rho = m/V$ becomes a division.

Exercise 13 — 13 · MET calories

Task. Estimate 8 kcal per minute for 30 minutes.

```
kcal_minute = 8
duree = 30
print(kcal_minute * duree)
```

Correction. The total is duration × expenditure.

Exercise 14 — 14 · DNA to RNA

Task. Replace thymine T with U in a sequence.

```
adn = "ATGCTT"
print(adn.replace("T", "U"))
```

Correction. replace transforms a string without an explicit loop.

Exercise 15 — 15 · Pendulum

Task. Calculate period with $T = 2\pi\sqrt{L/g}$.

```
from math import pi, sqrt
longueur = 1
g = 9.81
print(2 * pi * sqrt(longueur / g))
```

Correction. The math library provides pi and sqrt.

Conditions and loops

Worked examples

Example 1 — 01 · Evenness

Situation. A checkout distinguishes even and odd tickets.

```
n = 18
if n % 2 == 0: print("pair")
else: print("impair")
```

Explanation. Remainder 0 identifies an even number.

Example 2 — 02 · Threshold

Situation. A score determines success.

```
score = 14
if score ≥ 10: print("réussi")
else: print("à revoir")
```

Explanation. The comparison selects a branch.

Example 3 — 03 · Category

Situation. A temperature receives a label.

```
t = 28
if t < 10: cat = "froid"
elif t < 25: cat = "doux"
else: cat = "chaud"
```

Explanation. Thresholds are tested in order.

Example 4 — 04 · Counter

Situation. Count five passes.

```
compteur = 0
for i in range(5): compteur += 1
print(compteur)
```

Explanation. The counter changes each turn.

Example 5 — 05 · Accumulator

Situation. Total even numbers from 1 to 6.

```
total = 0
for n in range(1, 7):
    if n % 2 == 0: total += n
print(total)
```

Explanation. Only 2, 4 and 6 change total.

Example 6 — 06 · Water states

Situation. Classify water by temperature.

```
t = -4
if t < 0: print("solide")
elif t < 100: print("liquide")
else: print("gaz")
```

Explanation. Thresholds 0 and 100 form three intervals.

Example 7 — 07 · BMI

Situation. A learning diagnostic classifies BMI.

```
imc = 27.4
if imc < 18.5: c = "insuffisant"
elif imc < 25: c = "normal"
elif imc < 30: c = "surpoids"
else: c = "obésité"
print(c)
```

Explanation. Conditions are ordered from the lowest to the highest threshold.

Example 8 — 08 · Triangle

Situation. A triangle is identified from its three sides.

```
a, b, c = 3, 3, 5
if a == b == c: print("équilatéral")
elif a == b or a == c or b == c: print("isocèle")
else: print("scalène")
```

Explanation. Comparisons describe side equality.

Example 9 — 09 · pH

Situation. A solution is acidic, neutral, or basic.

```
ph = 8.2
if ph < 7: print("acide")
elif ph == 7: print("neutre")
else: print("basique")
```

Explanation. The three branches cover the three pH zones.

Example 10 — 10 · UV index

Situation. An app recommends protection from the UV index.

```
uv = 7
if uv ≤ 2: conseil = "faible"
elif uv ≤ 5: conseil = "modérée"
else: conseil = "forte"
print(conseil)
```

Explanation. An inclusive threshold is written with `<=`.

Solved exercises

Exercise 1 — 01 · Positive

Task. Test whether a number is positive.

```
n = -2
print(n > 0)
```

Correction. One comparison is enough.

Exercise 2 — 02 · Grade

Task. Display passed if grade `>= 10`.

```
note = 11
print("admis" if note ≥ 10 else "refusé")
```

Correction. Short conditional expression.

Exercise 3 — 03 · Multiple

Task. Test multiple of 3.

```
n = 12
print(n % 3 == 0)
```

Correction. Zero remainder means divisible.

Exercise 4 — 04 · Three cases

Task. Class a speed slow, normal, or fast.

```
v = 65
if v < 50: r="lente"
elif v ≤ 90: r="normale"
else: r="rapide"
```

Correction. elif handles the middle case.

Exercise 5 — 05 · Sum

Task. Add 1 to 5.

```
total = 0
for n in range(1, 6): total += n
```

Correction. The final total is 15.

Exercise 6 — 06 · Repeat

Task. Display a signal three times.

```
for _ in range(3): print("go")
```

Correction. `_` signals the index is unused.

Exercise 7 — 07 · While

Task. Count to 3 with while.

```
i = 1
while i ≤ 3:
    print(i); i += 1
```

Correction. The condition eventually becomes False.

Exercise 8 — 08 · Filter

Task. Count values > 10.

```
compte = 0
for n in [8, 12, 15]:
    if n > 10: compte += 1
```

Correction. The condition controls the counter.

Exercise 9 — 09 · Maximum

Task. Find maximum of three values.

```
maximum = 0
for n in [4, 9, 2]:
    if n > maximum: maximum = n
```

Correction. Each value is compared.

Exercise 10 — 10 · Stop

Task. Stop when 4 appears.

```
for n in [1, 4, 7]:  
    if n == 4: break
```

Correction. break interrupts the loop.

Exercise 11 — 11 · Archimedes

Task. Say whether an object floats when its density is lower than water.

```
densite = 0.8  
print("flotte" if densite < 1 else "coule")
```

Correction. The comparison chooses the physical outcome.

Exercise 12 — 12 · Leap year

Task. Test the leap-year rule.

```
annee = 2024  
print(annee % 400 == 0 or (annee % 4 == 0 and annee % 100 != 0))
```

Correction. and/or operators transcribe the logical rule.

Exercise 13 — 13 · Lens

Task. Class a lens from its power.

```
vergence = -2
print("convergente" if vergence > 0 else "divergente")
```

Correction. The sign of power is enough for this classification.

Exercise 14 — 14 · Blood pressure

Task. Flag systolic pressure above 140.

```
systolique = 145
if systolique > 140: print("à surveiller")
else: print("dans la zone")
```

Correction. The threshold triggers information, not a medical diagnosis.

Exercise 15 — 15 · Pythagorean theorem

Task. Check if three lengths form a right triangle.

```
a, b, c = 3, 4, 5
print(a*a + b*b == c*c)
```

Correction. The square of the hypotenuse is compared with the sum of squares.

Vectors and lists

Worked examples

Example 1 — 01 · Sales total

Situation. A shop wants the day total.

```
ventes = [12, 8, 15]
total = sum(ventes)
print(total)
```

Explanation. sum traverses the vector and returns 35.

Example 2 — 02 · Maximum temperature

Situation. A station records four temperatures and finds the peak.

```
temperatures = [18, 22, 19, 25]
maximum = temperatures[0]
for t in temperatures:
    if t > maximum: maximum = t
print(maximum)
```

Explanation. The maximum reference is replaced only by a larger value.

Example 3 — 03 · Filter passes

Situation. A class keeps grades that validate the skill.

```
notes = [8, 12, 15, 9]
reussites = []
for note in notes:
    if note ≥ 10: reussites.append(note)
print(reussites)
```

Explanation. A new list receives only values that pass.

Example 4 — 04 · Reverse a route

Situation. A route must be read in return order.

```
etapes = ["A", "B", "C"]
print(etapes[::-1])
```

Explanation. -1 slicing reads the list from right to left.

Example 5 — 05 · Checked mean

Situation. An empty vector must not cause division by zero.

```
notes = []
if notes:
    print(sum(notes) / len(notes))
else:
    print("Aucune note")
```

Explanation. The non-empty check protects the calculation.

Example 6 — 06 · Daily speed

Situation. A vector stores four daily distances.

```
distances = [12, 18, 15, 21]
for d in distances: print(d / 2)
```

Explanation. Traversal applies the same rule to every measurement.

Example 7 — 07 · Filtered temperatures

Situation. A station keeps temperatures above 20°C.

```
mesures = [18, 22, 25, 19]
chaudes = [t for t in mesures if t > 20]
print(chaudes)
```

Explanation. Filtering builds a more focused list.

Example 8 — 08 · Total energy

Situation. A vector contains energies measured during an experiment.

```
energies = [3.2, 4.1, 2.7]
print(sum(energies))
```

Explanation. `sum` gives the total without losing original values.

Example 9 — 09 · Peak position

Situation. Find the day corresponding to the maximum temperature.

```
temperatures = [19, 24, 22, 27]
print(temperatures.index(max(temperatures)))
```

Explanation. `index` turns the maximum value into a position.

Example 10 — 10 · Normalization

Situation. Measurements are scaled between 0 and 1.

```
mesures = [10, 20, 30]
mini, maxi = min(mesures), max(mesures)
print([(x-mini)/(maxi-mini) for x in mesures])
```

Explanation. The formula uses both vector extremes.

Solved exercises

Exercise 1 — 01 · Count

Task. Count even values.

```
valeurs = [2, 5, 8]
print(sum(n % 2 == 0 for n in valeurs))
```

Correction. Each comparison produces True or False, counted as 1 or 0.

Exercise 2 — 02 · Minimum

Task. Find the smallest item without min().

```
valeurs = [7, 3, 9]
minimum = valeurs[0]
for v in valeurs:
    if v < minimum: minimum = v
print(minimum)
```

Correction. Initialize with the first item to avoid an arbitrary value.

Exercise 3 — 03 · Indices

Task. Display each index and value.

```
for i in range(len(valeurs)):
    print(i, valeurs[i])
```

Correction. range traverses positions; valeurs[i] reads data.

Exercise 4 — 04 · Shift

Task. Add 2 to each item.

```
valeurs = [1, 4, 7]
modifiees = [v + 2 for v in valeurs]
```

Correction. A comprehension builds a new list.

Exercise 5 — 05 · Duplicates

Task. Remove duplicates while preserving order.

```
sortie = []  
for v in [2, 2, 3, 2]:  
    if v not in sortie: sortie.append(v)
```

Correction. in checks existence before append.

Exercise 6 — 06 · Rotation

Task. Move the first item to the end.

```
valeurs = ["A", "B", "C"]  
valeurs = valeurs[1:] + valeurs[:1]
```

Correction. Two slices rebuild the sequence.

Exercise 7 — 07 · Filtered sum

Task. Add only positive values.

```
total = 0  
for v in [-2, 4, -1, 6]:  
    if v > 0: total += v
```

Correction. The accumulator changes only for kept values.

Exercise 8 — 08 · Search index

Task. Find the first position of 9.

```
valeurs = [4, 9, 2]
for i, v in enumerate(valeurs):
    if v == 9: print(i); break
```

Correction. enumerate gives index and value; break stops at first.

Exercise 9 — 09 · Sorting

Task. Sort a temperature list.

```
temperatures = [23, 18, 25]
print(sorted(temperatures))
```

Correction. sorted creates a new sorted list.

Exercise 10 — 10 · Spread

Task. Calculate max-min spread.

```
valeurs = [4, 10, 7]
print(max(valeurs) - min(valeurs))
```

Correction. The spread describes the distance between extremes.

Exercise 11 — 11 · Average speed

Task. Calculate the mean of four speeds.

```
vitesses = [8, 10, 12, 10]
print(sum(vitesses) / len(vitesses))
```

Correction. The count comes from the list itself.

Exercise 12 — 12 · Negative values

Task. Count temperatures below zero.

```
temperatures = [-2, 4, -1, 3]
print(sum(t < 0 for t in temperatures))
```

Correction. Booleans can be added as 0 and 1.

Exercise 13 — 13 · Accumulation

Task. Calculate cumulative distance.

```
distances = [2, 3, 4]
cumul = 0
for d in distances:
    cumul += d
print(cumul)
```

Correction. The accumulator keeps state between turns.

Exercise 14 — 14 · Maximum index

Task. Find the position of the largest current.

```
courants = [2.1, 3.4, 2.8]
print(courants.index(max(courants)))
```

Correction. max then index link value and position.

Exercise 15 — 15 · Units

Task. Convert all lengths from cm to m.

```
longueurs = [30, 45, 120]
print([x / 100 for x in longueurs])
```

Correction. The comprehension applies the conversion to every item.

Dictionaries

Worked examples

Example 1 — 01 · Profile

Situation. A profile groups name, city, and age.

```
profil = {"nom": "Lina", "ville": "Oran", "age": 16}
print(profil["ville"])
```

Explanation. The city key gives explicit access.

Example 2 — 02 · Inventory

Situation. A sale decreases existing stock.

```
stock = {"clavier": 8, "souris": 12}
stock["clavier"] -= 1
```

Explanation. Assignment on a key updates its value.

Example 3 — 03 · Safe contact

Situation. A contact may have no phone.

```
contact = {"nom": "Ali"}
print(contact.get("telephone", "Absent"))
```

Explanation. get avoids KeyError and provides a fallback.

Example 4 — 04 · Subject grades

Situation. A dictionary maps each subject to a grade.

```
notes = {"maths": 15, "python": 17}
for matiere, note in notes.items():
    print(matiere, note)
```

Explanation. items gives the key-value pair.

Example 5 — 05 · Total inventory

Situation. An inventory calculates total quantity.

```
stock = {"livres": 4, "cahiers": 9}
print(sum(stock.values()))
```

Explanation. values extracts quantities without keys.

Example 6 — 06 · Scientific measurement

Situation. A dictionary links a value to its unit and sensor.

```
measure = {"valeur": 21.4, "unite": "°C", "capteur": "S1"}
print(measure["valeur"], measure["unite"])
```

Explanation. Keys make the value's context explicit.

Example 7 — 07 · Frequency by sport

Situation. A sports profile stores heart rate by activity.

```
fc = {"repos": 65, "course": 154}
print(fc.get("natation", "non mesurée"))
```

Explanation. get handles an activity not yet recorded.

Example 8 — 08 · Chemical composition

Situation. Mixture proportions are stored by element.

```
proportions = {"eau": 70, "sel": 5, "solvant": 25}
print(sum(proportions.values()))
```

Explanation. values makes it possible to check total proportions.

Example 9 — 09 · Count categories

Situation. A dictionary counts observations by category.

```
compte = {}
for etat in ["solide", "liquide", "liquide"]:
    compte[etat] = compte.get(etat, 0) + 1
print(compte)
```

Explanation. get provides zero on the first occurrence of a category.

Example 10 — 10 · Configuration

Situation. A protocol keeps named experimental parameters.

```
protocole = {"duree": 60, "temperature": 25, "agitation": True}
for cle, valeur in protocole.items(): print(cle, valeur)
```

Explanation. items traverses configuration without losing parameter names.

Solved exercises

Exercise 1 — 01 · Add

Task. Add an email key to a profile.

```
profil = {"nom": "Sara"}  
profil["email"] = "sara@mail.test"
```

Correction. A new key is created by assignment.

Exercise 2 — 02 · Delete

Task. Delete a temporary key.

```
donnees = {"nom": "Ali", "temp": True}  
del donnees["temp"]
```

Correction. del removes the requested key.

Exercise 3 — 03 · Count occurrences

Task. Count votes by candidate.

```
votes = {}  
for nom in ["A", "B", "A"]:  
    votes[nom] = votes.get(nom, 0) + 1
```

Correction. get(...,0) initializes a missing key.

Exercise 4 — 04 · Conditional lookup

Task. Display products with low stock.

```
stock = {"A": 2, "B": 8}
for nom, quantite in stock.items():
    if quantite < 5: print(nom)
```

Correction. The test applies to each value.

Exercise 5 — 05 · Controlled copy

Task. Copy only useful profile fields.

```
source = {"nom": "Lina", "age": 16, "secret": 1}
cible = {k: source[k] for k in ["nom", "age"]}
```

Correction. Selecting keys limits copied data.

Exercise 6 — 06 · Inversion

Task. Build a city → code dictionary.

```
codes = {"Oran": 31, "Alger": 16}
inverse = {code: ville for ville, code in codes.items()}
```

Correction. Each pair is reversed into a new structure.

Exercise 7 — 07 · Sum grades

Task. Calculate the mean of numeric values.

```
notes = {"A": 12, "B": 16}
print(sum(notes.values()) / len(notes))
```

Correction. values separates data from labels.

Exercise 8 — 08 · Missing key

Task. Display unknown when city is absent.

```
profil = {"nom": "Maya"}  
print(profil.get("ville", "inconnue"))
```

Correction. The default makes the program robust.

Exercise 9 — 09 · Grouping

Task. Group students by class.

```
classes = {}  
for nom, classe in [("A", "2A"), ("B", "2A")]:  
    classes.setdefault(classe, []).append(nom)
```

Correction. setdefault creates a list for a new class.

Exercise 10 — 10 · Report

Task. Display every field of a record.

```
fiche = {"id": "S1", "nom": "Lina"}  
for cle, valeur in fiche.items(): print(cle, ":", valeur)
```

Correction. items suits a complete report.

Exercise 11 — 11 · Score per student

Task. Create a score dictionary and display Lina's score.

```
scores = {"Lina": 16, "Ali": 12}  
print(scores["Lina"])
```

Correction. The key is the learner name.

Exercise 12 — 12 · Safe inventory

Task. Update stock only when the key exists.

```
stock = {"cahier": 12}
if "cahier" in stock: stock["cahier"] -= 1
```

Correction. `in` protects access before updating.

Exercise 13 — 13 · Subject mean

Task. Calculate the mean of grade values.

```
notes = {"maths": 14, "python": 18}
print(sum(notes.values()) / len(notes))
```

Correction. `values` extracts numeric data.

Exercise 14 — 14 · Invert key-value

Task. Create code-to-name from name-to-code.

```
noms = {"Na": "sodium", "O": "oxygene"}
inverse = {nom: code for code, nom in noms.items()}
```

Correction. The comprehension swaps the two roles.

Exercise 15 — 15 · Field presence

Task. Display a fallback if pressure is missing.

```
measure = {"temperature": 20}
print(measure.get("pression", "absente"))
```

Correction. `get` avoids `KeyError` and makes the program robust.

Records

Worked examples

Example 1 — 01 · Student record

Situation. A record gathers id, name, and grades.

```
eleve = {"id": "S1", "nom": "Maya", "notes": [12, 15]}
```

Explanation. The schema associates each field with a coherent type.

Example 2 — 02 · Grade validation

Situation. A grade must stay between 0 and 20.

```
note = 18
valide = 0 ≤ note ≤ 20
print(valide)
```

Explanation. A chained comparison checks both bounds.

Example 3 — 03 · Record list

Situation. Several students are traversed for a report.

```
eleves = [{"nom": "A", "note": 12}, {"nom": "B", "note": 8}]
for eleve in eleves: print(eleve["nom"])
```

Explanation. Each item is an independent record.

Example 4 — 04 · Computed field

Situation. A record can receive a calculated mean.

```
eleve = {"nom": "Lina", "notes": [14, 16]}
eleve["moyenne"] = sum(eleve["notes"]) / len(eleve["notes"])
```

Explanation. The computed field complements the record.

Example 5 — 05 · Search by id

Situation. A record is found by its identifier.

```
elevés = [{"id": "S1"}, {"id": "S2"}]
for e in élevés:
    if e["id"] == "S2": print(e)
```

Explanation. Traversal conceptually stops when the correct record is found.

Example 6 — 06 · Athlete

Situation. A record groups a runner's name, distance, and time.

```
athlete = {"nom": "Nora", "distance": 5, "temps": 28}
print(athlete["distance"] / (athlete["temps"] / 60))
```

Explanation. Fields allow pace calculation from a record.

Example 7 — 07 · Experiment

Situation. An experiment stores temperature, mass, and result.

```
experience = {"temperature": 25, "masse": 12.4, "resultat": "stable"}  
if experience["resultat"] == "stable": print("valider")
```

Explanation. The result field drives the validation decision.

Example 8 — 08 · File validation

Situation. A file is accepted if required fields are present.

```
dossier = {"nom": "Samir", "email": "samir@example.com"}  
obligatoires = ["nom", "email"]  
print(all(cle in dossier for cle in obligatoires))
```

Explanation. all checks every expected key.

Example 9 — 09 · Measurement records

Situation. Several records represent sensor readings.

```
mesures = [{"id": 1, "valeur": 18}, {"id": 2, "valeur": 22}]  
print([m["valeur"] for m in mesures])
```

Explanation. A list of dictionaries forms a simple table.

Example 10 — 10 · Report

Situation. A complete record produces a readable report.

```
eleve = {"prenom": "Lina", "notes": [14, 16]}  
moy = sum(eleve["notes"]) / len(eleve["notes"])  
print(f"{eleve['prenom']}: {moy}")
```

Explanation. The calculation uses notes then the name presents the result.

Solved exercises

Exercise 1 — 01 · Book

Task. Create a book record with title and year.

```
livre = {"titre": "Python", "annee": 2026}
```

Correction. Fields describe one entity.

Exercise 2 — 02 · Validation

Task. Reject a record if its name is empty.

```
fiche = {"nom": ""}  
print(bool(fiche.get("nom")))
```

Correction. bool turns empty text into False.

Exercise 3 — 03 · Record mean

Task. Add a mean to each student.

```
for e in eleves: e["moyenne"] = sum(e["notes"]) / len(e["notes"])
```

Correction. The same processing is applied to each record.

Exercise 4 — 04 · Filter records

Task. Keep passed students.

```
admis = [e for e in eleves if e["moyenne"] ≥ 10]
```

Correction. The filter uses a computed field.

Exercise 5 — 05 · Identifier

Task. Check that two records do not share an id.

```
ids = [e["id"] for e in eleves]
print(len(ids) == len(set(ids)))
```

Correction. Comparing lengths detects a duplicate.

Exercise 6 — 06 · Sensor

Task. Create a sensor record with unit and measure.

```
capteur = {"nom": "T1", "unite": "°C", "mesure": 21.4}
```

Correction. Fields document the measurement.

Exercise 7 — 07 · Bounds

Task. Validate a measure in a range.

```
m = 21.4
print(0 ≤ m ≤ 50)
```

Correction. Validation protects the next analysis.

Exercise 8 — 08 · Table

Task. Traverse names in a record table.

```
for e in eleves: print(e.get("nom", "inconnu"))
```

Correction. get avoids stopping if a field is missing.

Exercise 9 — 09 · ID lookup

Task. Return the record matching an id.

```
trouve = next((e for e in eleves if e["id"] == "S2"), None)
```

Correction. next returns None if no record matches.

Exercise 10 — 10 · Summary

Task. Count valid records.

```
valides = [e for e in eleves if e.get("moyenne", 0) ≥ 10]  
print(len(valides))
```

Correction. The retained record count becomes an indicator.

Exercise 11 — 11 · Pressure record

Task. Create a record with systolic, diastolic, and date.

```
mesure = {"systolique": 120, "diastolique": 80, "date": "2026-06-01"}
```

Correction. The three fields describe one measurement.

Exercise 12 — 12 · Filter records

Task. Keep measurements whose value exceeds 20.

```
mesures = [{"valeur": 18}, {"valeur": 24}]  
print([m for m in mesures if m["valeur"] > 20])
```

Correction. The filter targets a record field.

Exercise 13 — 13 · Completeness

Task. Test whether each student has an identifier.

```
eleves = [{"id": 1}, {"id": 2}]
print(all("id" in e for e in eleves))
```

Correction. Validation traverses the whole record list.

Exercise 14 — 14 · Update

Task. Increase a product stock after delivery.

```
produit = {"nom": "Tube", "stock": 4}
produit["stock"] += 6
```

Correction. Only the target field is modified.

Exercise 15 — 15 · Search identifier

Task. Find the record whose id is 2.

```
eleves = [{"id": 1, "nom": "Ali"}, {"id": 2, "nom": "Nora"}]
print(next(e for e in eleves if e["id"] == 2))
```

Correction. next retrieves the first record matching the test.

Stacks

Worked examples

Example 1 — 01 · Undo

Situation. An editor undoes the latest action.

```
pile = ["titre", "image"]
print(pile.pop())
```

Explanation. pop removes image: last in, first out.

Example 2 — 02 · History

Situation. A browser goes back.

```
historique = ["accueil", "cours", "quiz"]
page = historique.pop()
print(page, historique[-1])
```

Explanation. The new current page is the remaining top.

Example 3 — 03 · Parentheses

Situation. A stack checks opening parentheses.

```
pile = []
for symbole in "(()":
    if symbole == "(": pile.append(symbole)
print(len(pile))
```

Explanation. Each opening is pushed to be closed later.

Example 4 — 04 · Empty

Situation. Undo must not pop an empty stack.

```
pile = []
if pile: print(pile.pop())
else: print("Rien à annuler")
```

Explanation. The check protects pop.

Example 5 — 05 · Two undos

Situation. Two recent actions are undone.

```
actions = ["A", "B", "C"]
for _ in range(2): print(actions.pop())
```

Explanation. The loop pops C then B.

Example 6 — 06 · Parentheses

Situation. A stack helps check opening and closing parentheses.

```
pile = []
for symbole in "(":
    if symbole == "(": pile.append(symbole)
print(len(pile))
```

Explanation. Each opening is pushed and awaits a closing symbol.

Example 7 — 07 · Reverse DNA

Situation. A sequence is read in reverse using a stack.

```
pile = list("ATGC")
retour = ""
while pile: retour += pile.pop()
print(retour)
```

Explanation. The last character entered becomes the first read.

Example 8 — 08 · Scientific history

Situation. Software undoes the last transformation applied to a measurement.

```
historique = ["filtre", "normalisation", "arrondi"]
print("annuler", historique.pop())
```

Explanation. pop removes the most recent operation.

Example 9 — 09 · Nested calls

Situation. A conceptual stack represents calling functions.

```
appels = ["main", "analyser", "moyenne"]
print(appels[-1])
appels.pop()
```

Explanation. The top represents the most recent active call.

Example 10 — 10 · Postfix expression

Situation. A stack stores operands before an operation.

```
pile = [3, 4]
b = pile.pop(); a = pile.pop()
print(a + b)
```

Explanation. The two latest items are retrieved for the calculation.

Solved exercises

Exercise 1 — 01 · Push

Task. Push three operations.

```
pile = []
for action in ["A", "B", "C"]: pile.append(action)
```

Correction. append places each action on top.

Exercise 2 — 02 · Pop

Task. Pop the top if possible.

```
if pile: resultat = pile.pop()
```

Correction. if protects removal.

Exercise 3 — 03 · Peek

Task. Read the top without removing it.

```
sommet = pile[-1] if pile else None
```

Correction. Index -1 reads without changing.

Exercise 4 — 04 · Size

Task. Display remaining actions.

```
print(len(pile))
```

Correction. len measures the current stack.

Exercise 5 — 05 · Empty

Task. Empty a stack with while.

```
while pile: pile.pop()
```

Correction. Each turn removes a top item.

Exercise 6 — 06 · Reverse

Task. Reverse a word with a stack.

```
pile = list("PY")
print(pile.pop() + pile.pop())
```

Correction. Outputs follow reverse order.

Exercise 7 — 07 · Trace

Task. Display the stack after each action.

```
pile = []
for x in [1,2]:
    pile.append(x); print(pile)
```

Correction. The trace makes state visible.

Exercise 8 — 08 · Latest

Task. Retrieve the last visited page.

```
page = historique[-1] if historique else "aucune"
```

Correction. The empty case needs an answer.

Exercise 9 — 09 · Limit

Task. Limit a stack to three items.

```
if len(pile) < 3: pile.append("nouveau")
```

Correction. The condition controls capacity.

Exercise 10 — 10 · Function

Task. Write a safe pop function.

```
def depiler(pile):  
    return pile.pop() if pile else None
```

Correction. return None describes absence.

Exercise 11 — 11 · Safe undo

Task. Write annuler(stack) returning None if empty.

```
def annuler(pile):  
    return pile.pop() if pile else None
```

Correction. The check protects pop on an empty stack.

Exercise 12 — 12 · Two-action history

Task. Undo the two latest actions.

```
actions = ["A", "B", "C"]  
for _ in range(2): print(actions.pop())
```

Correction. The loop repeats popping.

Exercise 13 — 13 · Palindrome

Task. Use a stack to compare a word with its reverse.

```
mot = "kayak"  
pile = list(mot)  
print(mot == "".join(pile.pop() for _ in range(len(pile))))
```

Correction. Reverse reading is compared with the original word.

Exercise 14 — 14 · Peek

Task. Read the top without popping.

```
pile = [10, 20, 30]
print(pile[-1])
```

Correction. Index -1 observes without modifying.

Exercise 15 — 15 · Empty

Task. Empty a stack while displaying each item.

```
pile = [1, 2, 3]
while pile: print(pile.pop())
```

Correction. while stops when the stack becomes empty.

Queues

Worked examples

Example 1 — 01 · Printing

Situation. A printer processes documents by arrival order.

```
file = ["A.pdf", "B.pdf"]
print(file.pop(0))
```

Explanation. pop(0) removes the oldest document.

Example 2 — 02 · Desk

Situation. A desk serves the first person and keeps the rest.

```
file = ["Nora", "Ali", "Maya"]  
servie = file.pop(0)
```

Explanation. The queue keeps Ali then Maya.

Example 3 — 03 · Empty

Situation. A system does not serve an empty queue.

```
file = []  
if file: print(file.pop(0))  
else: print("Attente vide")
```

Explanation. The check avoids IndexError.

Example 4 — 04 · Counter

Situation. A server counts processed requests.

```
file = ["A", "B"]  
compteur = 0  
while file:  
    file.pop(0); compteur += 1  
print(compteur)
```

Explanation. The counter measures services, not final size.

Example 5 — 05 · Enqueue

Situation. A new request arrives after existing ones.

```
file = ["A", "B"]
file.append("C")
print(file)
```

Explanation. append adds at the end and preserves order.

Example 6 — 06 · Samples

Situation. A lab processes samples in arrival order.

```
file = ["S1", "S2", "S3"]
print(file.pop(0))
```

Explanation. The oldest sample is processed first.

Example 7 — 07 · Marathon

Situation. Runners are called to control in order.

```
file = ["dossard 12", "dossard 18"]
file.append("dossard 24")
print(file.pop(0))
```

Explanation. append adds behind; pop(0) serves from the front.

Example 8 — 08 · Sensor messages

Situation. A queue holds messages received before analysis.

```
messages = ["T=20", "T=21"]
while messages:
    print("analyse", messages.pop(0))
```

Explanation. The loop empties the queue without reversing messages.

Example 9 — 09 · Patient service

Situation. A reception processes requests in registration order.

```
demandes = ["A", "B", "C"]
prochaine = demandes.pop(0)
print(prochaine)
```

Explanation. The first request remains first in a FIFO queue.

Example 10 — 10 · Documents

Situation. A printer keeps received file order.

```
documents = []
for nom in ["a.pdf", "b.pdf"]: documents.append(nom)
print(documents.pop(0))
```

Explanation. The queue represents service order.

Solved exercises

Exercise 1 — 01 · Enqueue

Task. Add two tickets.

```
file = []
file.append("T1"); file.append("T2")
```

Correction. Arrivals go at the end.

Exercise 2 — 02 · Serve

Task. Serve the first ticket.

```
ticket = file.pop(0) if file else None
```

Correction. The condition protects removal.

Exercise 3 — 03 · Loop

Task. Process a whole queue.

```
while file:  
    print(file.pop(0))
```

Correction. Each removal respects FIFO.

Exercise 4 — 04 · Size

Task. Display waiting count.

```
print(len(file))
```

Correction. len measures current waiting.

Exercise 5 — 05 · Priority

Task. Add an urgent request at the front.

```
file.insert(0, "URGENT")
```

Correction. A priority deliberately changes FIFO.

Exercise 6 — 06 · Summary

Task. Count served customers.

```
servis = 0
while file:
    file.pop(0); servis += 1
```

Correction. The counter is independent of the queue.

Exercise 7 — 07 · Function

Task. Write `serve(file)` returning `None` when empty.

```
def servir(file):
    return file.pop(0) if file else None
```

Correction. A function makes the rule reusable.

Exercise 8 — 08 · Rotation

Task. Move the first customer to the end.

```
file.append(file.pop(0))
```

Correction. Removal is followed by an add.

Exercise 9 — 09 · Waiting

Task. Display mean waiting time from a list.

```
temps = [4, 7, 5]
print(sum(temps) / len(temps))
```

Correction. The time vector describes the queue.

Exercise 10 — 10 · Two queues

Task. Merge two queues preserving internal order.

```
file1 = ["A","B"]; file2 = ["C","D"]  
file = file1 + file2
```

Correction. Concatenation places the second queue after the first.

Exercise 11 — 11 · Enqueue

Task. Add a ticket to a queue.

```
file = ["T1"]  
file.append("T2")
```

Correction. `append` places the new ticket behind.

Exercise 12 — 12 · Serve five

Task. Serve five requests with `while`.

```
file = [1, 2, 3, 4, 5]  
while file: print(file.pop(0))
```

Correction. Each removal respects FIFO.

Exercise 13 — 13 · Waiting time

Task. Add processing times in a queue.

```
file = [3, 4, 2]  
total = 0  
while file: total += file.pop(0)  
print(total)
```

Correction. The accumulator follows sequential processing.

Exercise 14 — 14 · Simple priority

Task. Place an emergency at the front.

```
file = ["normal"]
file.insert(0, "urgent")
print(file.pop(0))
```

Correction. insert(0) intentionally changes order for priority.

Exercise 15 — 15 · Length

Task. Display the number of requests waiting.

```
file = ["A", "B", "C"]
file.pop(0)
print(len(file))
```

Correction. len measures the current state after service.

Trees

Worked examples

Example 1 — 01 · Root

Situation. A project tree starts with a root.

```
arbre = {"valeur": "Projet", "enfants": []}
print(arbre["valeur"])
```

Explanation. The root is the first node.

Example 2 — 02 · Leaves

Situation. A leaf folder has no children.

```
noeud = {"valeur": "README", "enfants": []}  
print(noeud["enfants"] == [])
```

Explanation. The empty list is the stopping case.

Example 3 — 03 · Traversal

Situation. Visit a parent then its children.

```
def visiter(n):  
    print(n["valeur"])  
    for enfant in n["enfants"]: visiter(enfant)
```

Explanation. The same function handles each subtree.

Example 4 — 04 · Height

Situation. Height counts branch levels.

```
def hauteur(n):  
    if not n["enfants"]: return 1  
    return 1 + max(hauteur(e) for e in n["enfants"])
```

Explanation. A leaf has height 1; otherwise add one level.

Example 5 — 05 · Search

Situation. Search for a value in a hierarchy.

```
def contient(n, cible):  
    return n["valeur"] == cible or any(contient(e, cible) for e in n["enfants"])
```

Explanation. Search stops when a branch returns True.

Example 6 — 06 · Living classification

Situation. A hierarchy links kingdom, class, and species.

```
arbre = {"valeur": "Animal", "enfants": [{"valeur": "Mammifère", "enfants": []}]}
print(arbre["valeur"])
```

Explanation. The root holds the general level and children hold sublevels.

Example 7 — 07 · Folders

Situation. A folder contains subfolders and files.

```
racine = {"nom": "cours", "enfants": [{"nom": "python", "enfants": []}, {"nom": "java", "enfants": []}]}
print(len(racine["enfants"]))
```

Explanation. The children list holds direct branches.

Example 8 — 08 · Decision

Situation. A decision tree separates profiles by a threshold.

```
def decision(note):
    return {"valeur": "reussi" if note ≥ 10 else "a_reprendre", "enfants": []}
print(decision(14))
```

Explanation. A conditional branch produces a leaf.

Example 9 — 09 · Count leaves

Situation. Count final results in a tree.

```
def feuilles(n):
    if not n["enfants"]: return 1
    return sum(feuilles(e) for e in n["enfants"])
```

Explanation. The stopping case is a node without children.

Example 10 — 10 · Depth traversal

Situation. A report displays a hierarchy with its level.

```
def afficher(n, niveau=0):
    print("  " * niveau + n["valeur"])
    for e in n["enfants"]: afficher(e, niveau + 1)
```

Explanation. Level controls trace indentation.

Solved exercises

Exercise 1 — 01 · Build

Task. Create a root with two leaves.

```
arbre = {"valeur": "A", "enfants": [{"valeur": "B", "enfants": []}, {"valeur": "C", "enfants": []}]}
```

Correction. Each child follows the same model.

Exercise 2 — 02 · Display

Task. Display all values depth-first.

```
def afficher(n):
    print(n["valeur"])
    for e in n["enfants"]: afficher(e)
```

Correction. The recursive call descends every branch.

Exercise 3 — 03 · Count

Task. Count tree nodes.

```
def compter(n):  
    return 1 + sum(compter(e) for e in n["enfants"])
```

Correction. 1 counts the current node.

Exercise 4 — 04 · Leaf

Task. Test whether a node is a leaf.

```
def feuille(n): return len(n["enfants"]) == 0
```

Correction. No child means leaf.

Exercise 5 — 05 · Height

Task. Calculate recursive height.

```
def h(n):  
    return 1 if not n["enfants"] else 1 + max(h(e) for e in n["enfants"])
```

Correction. The stopping case prevents infinite recursion.

Exercise 6 — 06 · Search

Task. Search for value “Courses”.

```
trouve = contient(arbre, "Cours")
```

Correction. or combines current test and children.

Exercise 7 — 07 · Levels

Task. Display with indentation.

```
def afficher(n, niveau=0):  
    print("  " * niveau + n["valeur"])
```

Correction. level changes at each descent.

Exercise 8 — 08 · Leaves

Task. Count only leaves.

```
def feuilles(n):  
    if not n["enfants"]: return 1  
    return sum(feuilles(e) for e in n["enfants"] )
```

Correction. The leaf case returns 1.

Exercise 9 — 09 · Filter

Task. Return nodes whose name starts with C.

```
def ok(n): return n["valeur"].startswith("C")
```

Correction. startswith formalizes the criterion.

Exercise 10 — 10 · Empty tree

Task. Protect a function against None.

```
def afficher(n):  
    if n is None: return  
    print(n["valeur"])
```

Correction. The check makes traversal robust.

Exercise 11 — 11 · Create a node

Task. Create a node with value and children.

```
noeud = {"valeur": "racine", "enfants": []}
```

Correction. The same schema can be reused for every node.

Exercise 12 — 12 · Count nodes

Task. Recursively count all nodes.

```
def compter(n):  
    return 1 + sum(compter(e) for e in n["enfants"])
```

Correction. The current node counts as 1, then children are added.

Exercise 13 — 13 · Search value

Task. Search for a value in a tree.

```
def contient(n, cible):  
    return n["valeur"] == cible or any(contient(e, cible) for e in n["enfants"])
```

Correction. or and any stop once a branch succeeds.

Exercise 14 — 14 · Height

Task. Calculate tree height.

```
def hauteur(n):  
    if not n["enfants"]: return 0  
    return 1 + max(hauteur(e) for e in n["enfants"])
```

Correction. A leaf has height zero.

Exercise 15 — 15 · Display leaves

Task. Display leaves only.

```
def afficher_feuilles(n):  
    if not n["enfants"]: print(n["valeur"])  
    for e in n["enfants"]: afficher_feuilles(e)
```

Correction. Recursion descends to nodes without children.

Introduction to statistics

Worked examples

Example 1 — 01 · Mean

Situation. A class summarizes five grades.

```
notes = [8, 12, 14, 15, 19]  
print(sum(notes) / len(notes))
```

Explanation. The mean is 13.6.

Example 2 — 02 · Median

Situation. Find the central value of sorted times.

```
temps = [18, 25, 20, 22, 35]
tries = sorted(temps)
print(tries[len(tries)//2])
```

Explanation. Sorting gives [18,20,22,25,35]; median 22.

Example 3 — 03 · Range

Situation. A team measures extreme spread.

```
mesures = [4, 8, 10]
print(max(mesures) - min(mesures))
```

Explanation. The range is 6.

Example 4 — 04 · Minimum and maximum

Situation. Measurements are checked before publication.

```
mesures = [12.5, 9.8, 15.2]
print(min(mesures), max(mesures))
```

Explanation. min and max give observed bounds.

Example 5 — 05 · Outlier

Situation. A high value is compared with the mean.

```
notes = [10, 10, 10, 20]
print(sum(notes)/len(notes))
```

Explanation. 20 pulls the mean up; provide context.

Example 6 — 06 · Heart rate

Situation. A heart-rate series is summarized by mean and range.

```
fc = [62, 70, 78, 66]
print(sum(fc)/len(fc), max(fc)-min(fc))
```

Explanation. Two indicators describe level and simple spread.

Example 7 — 07 · Bacterial population

Situation. A series follows growth by day.

```
pop = [100, 120, 150, 190]
variations = [pop[i+1]-pop[i] for i in range(len(pop)-1)]
print(variations)
```

Explanation. Successive differences describe the dynamics.

Example 8 — 08 · Simple quartiles

Situation. Sorted measures are split into two halves.

```
valeurs = sorted([2, 4, 5, 8, 10, 12])
print(valeurs[:3], valeurs[3:])
```

Explanation. Splitting gives a first reading of spread.

Example 9 — 09 · Frequencies

Situation. Count grade occurrences in a class.

```
notes = [10, 12, 10, 15, 12, 10]
print({n: notes.count(n) for n in set(notes)})
```

Explanation. The absolute frequency of each value is observable.

Example 10 — 10 · Interpretation

Situation. A mean is compared to a learning target.

```
moyenne = 13.4
objectif = 12
print("objectif atteint" if moyenne ≥ objectif else "notion à retravailler")
```

Explanation. The calculation becomes an interpreted decision.

Solved exercises

Exercise 1 — 01 · Mean

Task. Calculate mean of [10,12,14].

```
notes = [10,12,14]
print(sum(notes)/len(notes))
```

Correction. The mean is 12.

Exercise 2 — 02 · Even median

Task. Calculate median of four values.

```
valeurs = sorted([2, 8, 4, 6])
mediane = (valeurs[1] + valeurs[2]) / 2
```

Correction. For an even count, average the two centers.

Exercise 3 — 03 · Range

Task. Calculate max-min.

```
valeurs = [3, 11, 7]
print(max(valeurs)-min(valeurs))
```

Correction. The range is 8.

Exercise 4 — 04 · Clean data

Task. Remove a None before calculating.

```
valeurs = [4, None, 6]
propres = [v for v in valeurs if v is not None]
```

Correction. Calculate only on numeric values.

Exercise 5 — 05 · Count

Task. Count list size.

```
donnees = [2, 4, 6, 8]
print(len(donnees))
```

Correction. len gives the count.

Exercise 6 — 06 · Above mean

Task. Keep values above the mean.

```
m = sum(notes)/len(notes)
print([n for n in notes if n > m])
```

Correction. Calculate m before filtering.

Exercise 7 — 07 · Compare

Task. Compare two means.

```
a = [10, 12]; b = [14, 16]
print(sum(a)/len(a) < sum(b)/len(b))
```

Correction. Compare indicators rather than raw lists.

Exercise 8 — 08 · Distribution

Task. Count grades by value.

```
notes = [10, 10, 12]
frequencies = {}
for n in notes: frequencies[n] = frequencies.get(n, 0) + 1
```

Correction. The dictionary becomes a frequency table.

Exercise 9 — 09 · Report

Task. Display mean, median, and range.

```
v = sorted([4, 8, 10, 12, 16])
print(sum(v)/len(v), v[len(v)//2], max(v)-min(v))
```

Correction. Three indicators describe the series better than one.

Exercise 10 — 10 · Interpret

Task. Write a sentence if mean exceeds 10.

```
m = sum(notes)/len(notes)
print("niveau global satisfaisant" if m ≥ 10 else "à retravailler")
```

Correction. The calculation becomes an explicit decision.

Exercise 11 — 11 · Weighted mean

Task. Calculate a weighted mean.

```
notes = [12, 16]
coeffs = [1, 2]
print(sum(n*c for n,c in zip(notes, coeffs)) / sum(coeffs))
```

Correction. zip pairs each grade with its coefficient.

Exercise 12 — 12 · Growth rate

Task. Calculate growth from 100 to 125.

```
ancien, nouveau = 100, 125
print((nouveau - ancien) / ancien * 100)
```

Correction. The rate compares change with starting value.

Exercise 13 — 13 · Odd median

Task. Find the median of five measurements.

```
valeurs = sorted([9, 2, 7, 4, 5])
print(valeurs[len(valeurs)//2])
```

Correction. Sorting makes the central index meaningful.

Exercise 14 — 14 · Missing data

Task. Ignore None values before averaging.

```
valeurs = [10, None, 14]
propres = [v for v in valeurs if v is not None]
print(sum(propres)/len(propres))
```

Correction. Data preparation precedes calculation.

Exercise 15 — 15 · Compare groups

Task. Compare the mean of two grade groups.

```
a, b = [12, 14], [15, 16]
ma, mb = sum(a)/len(a), sum(b)/len(b)
print("B" if mb > ma else "A")
```

Correction. Both means are calculated before comparison.

Functions and memory

Worked examples

Example 1 — 01 · Greet

Situation. A function builds a personalized message.

```
def saluer(nom):
    return "Bonjour " + nom
print(saluer("Lina"))
```

Explanation. The local parameter receives Lina.

Example 2 — 02 · Square

Situation. A reusable calculation produces a square.

```
def carre(n): return n * n
print(carre(6))
```

Explanation. return sends 36 to the caller.

Example 3 — 03 · VAT

Situation. A shop calculates a tax-included price.

```
def ttc(prix, taux=0.2): return prix * (1+taux)
print(ttc(100))
```

Explanation. The default rate is used when absent.

Example 4 — 04 · Validation

Situation. A function protects a score.

```
def valide(note): return 0 ≤ note ≤ 20
print(valide(18))
```

Explanation. The function returns a boolean decision.

Example 5 — 05 · Composition

Situation. Two functions convert then categorize temperature.

```
def c_to_f(c): return c * 9 / 5 + 32
def chaud(f): return f ≥ 77
print(chaud(c_to_f(25)))
```

Explanation. The first return value enters the second function.

Example 6 — 06 · Circle area

Situation. A function calculates area for several radii.

```
from math import pi
def aire_disque(r):
    return pi * r ** 2
print(aire_disque(3))
```

Explanation. The radius is local to the calculation and return makes the value reusable.

Example 7 — 07 · Speed

Situation. A function calculates average speed with explicit units.

```
def vitesse(distance_km, temps_h):
    return distance_km / temps_h
print(vitesse(42, 2))
```

Explanation. Two arguments enter, one numeric value leaves.

Example 8 — 08 · Energy

Situation. A model calculates gravitational energy.

```
def energie(masse, g, hauteur):
    return masse * g * hauteur
print(energie(2, 9.81, 3))
```

Explanation. The function isolates the scientific formula.

Example 9 — 09 · Concentration

Situation. A solution is described by dissolved mass and volume.

```
def concentration(masse, volume):  
    return masse / volume  
print(concentration(12, 0.5))
```

Explanation. The contract assumes a non-zero volume.

Example 10 — 10 · Pressure

Situation. A function turns a value into an informative category.

```
def zone_pression(valeur):  
    if valeur < 90: return "basse"  
    if valeur ≤ 120: return "habituelle"  
    return "élevée"  
print(zone_pression(128))
```

Explanation. A function can combine conditions and return.

Solved exercises

Exercise 1 — 01 · Even

Task. Write `is_even`.

```
def est_pair(n): return n % 2 == 0
```

Correction. The function returns True or False.

Exercise 2 — 02 · Mean

Task. Write `mean(a,b)`.

```
def moyenne(a,b): return (a+b)/2
```

Correction. Parentheses protect the sum.

Exercise 3 — 03 · Maximum

Task. Write `larger(a,b)`.

```
def plus_grand(a,b): return a if a ≥ b else b
```

Correction. The choice is returned, not only displayed.

Exercise 4 — 04 · Default

Task. Create a discount function defaulting to 10%.

```
def remise(prix, taux=10): return prix*(1-taux/100)
```

Correction. `taux` has a default value.

Exercise 5 — 05 · Text

Task. Return a complete sentence.

```
def phrase(nom, ville): return f"{nom} habite à {ville}."
```

Correction. f-string assembles variables.

Exercise 6 — 06 · Conversion

Task. Convert minutes to seconds.

```
def secondes(minutes): return minutes*60
```

Correction. A function can stay highly focused.

Exercise 7 — 07 · Condition

Task. Return an age category.

```
def categorie(age):  
    if age < 18: return "mineur"  
    return "majeur"
```

Correction. Each branch returns immediately.

Exercise 8 — 08 · List

Task. Return list sum.

```
def total(valeurs): return sum(valeurs)
```

Correction. The list is an argument.

Exercise 9 — 09 · Test

Task. Test with two calls.

```
assert est_pair(4)  
assert not est_pair(5)
```

Correction. assert checks an expectation automatically.

Exercise 10 — 10 · Documentation

Task. Add a docstring.

```
def carre(n):  
    """Renvoie le carré de n."""  
    return n*n
```

Correction. The docstring explains the contract.

Exercise 11 — 11 · Max heart rate

Task. Create `fcm(age)` with $220 - \text{age}$.

```
def fcm(age):  
    return 220 - age
```

Correction. `age` is used only inside the function.

Exercise 12 — 12 · Kelvin

Task. Create `celsius_to_kelvin©`.

```
def celsius_vers_kelvin(c):  
    return c + 273.15
```

Correction. The function returns a new unit.

Exercise 13 — 13 · Ohm law

Task. Create `current(u, r)`.

```
def intensite(u, r):  
    return u / r
```

Correction. The U/R calculation is isolated and testable.

Exercise 14 — 14 · Parity

Task. Create `is_even(n)` with a boolean.

```
def est_pair(n):  
    return n % 2 == 0
```

Correction. The result is directly reusable in if.

Exercise 15 — 15 · Two calls

Task. Test speed on two routes.

```
def vitesse(d, t): return d / t  
assert vitesse(100, 2) == 50  
assert vitesse(30, 1.5) == 20
```

Correction. assert turns expectations into readable tests.

Suggested learning routine

Read one example, predict its output, run it, then explain the correction in your own words. For each exercise, first write a hypothesis, test boundary cases, and only then compare your solution with the correction.

Author

M. Haithem BERKANE · Berkane Lab